



MEMORIA CUARTA PRACTICA: PROCESADOR DE LENGUAJE EN HASKELL

Acedo Blanco, Alvaro
Czepiel Babiarz, Daniel

24 de junio de 2026

Índice

1. Diseño del analizador léxico	2
1.1. Tokens	2
1.2. Gramática	2
1.3. Diagrama del autómata	3
1.4. Descripción de las Acciones Semánticas	3
1.5. Errores	4
2. Diseño del analizador sintáctico	5
2.1. Gramática	5
2.2. Tabla Predictiva y Demostración LL(1)	5
2.3. Gestión de Errores Sintácticos	6
3. Diseño del analizador semántico	8
3.1. Errores	10
4. Diseño de la Tabla de Símbolos	11
4.1. Organización y Ámbitos	11
4.2. Estructura de la Información	11
5. Diseño del Gestor de Errores	12
5.1. Seguimiento de Líneas	12
5.2. Estrategia de Recuperación de Errores	12

$$\begin{array}{l} A \rightarrow dA \mid .B \mid \lambda \\ B \rightarrow dC \\ C \rightarrow dC \mid \lambda \\ D \rightarrow cD \mid _D \mid dD \mid \lambda \\ E \rightarrow c_1E \mid \setminus F \mid " \\ F \rightarrow nE \mid tE \\ G \rightarrow = \mid \lambda \\ H \rightarrow \& \\ I \rightarrow - \mid \lambda \\ J \rightarrow /K \\ K \rightarrow c_2K \mid CRS \mid EOF \\ \} \end{array}$$

1.3. Diagrama del autómata

1.4. Descripción de las Acciones Semánticas

G1: Generar Token $\langle \text{eof}, - \rangle$

G2: Generar Token $\langle \text{not}, - \rangle$

G3: Generar Token $\langle \text{and}, - \rangle$

G4: Generar Token $\langle \text{asignacion}, - \rangle$

G5: Generar Token $\langle \text{llaveApertura}, - \rangle$

G6: Generar Token $\langle \text{llaveCierre}, - \rangle$

G7: Generar Token $\langle \text{coma}, - \rangle$

G8: Generar Token $\langle \text{parentesisApertura}, - \rangle$

G9: Generar Token $\langle \text{parentesisCierre}, - \rangle$

G10: Generar Token $\langle \text{suma}, - \rangle$

G11: Generar Token $\langle \text{resta}, - \rangle$

G12: Generar Token $\langle \text{mayor}, - \rangle$

G13: Generar Token $\langle \text{mayorIgual}, - \rangle$

G14: Generar Token $\langle \text{autodecremento}, - \rangle$

G15: **if** $p \neq \text{null}$ **then**
 Generar Token $\langle \text{PR_p}, - \rangle$
 else
 $p = \text{BuscarTS}(\text{lexema})$
 if $p == \text{null}$ **then**
 $p = \text{InsertarTS}(\text{lexema})$
 Generar Token $\langle \text{identificador}, p \rangle$

G16: **if** $\text{cont} < 65$ **then**
 Generar Token $\langle \text{cadena}, \text{lexema} \rangle$
 else
 Error(28)

G17: **if** $2^{15} > \text{num}$ **then**
 Generar Token $\langle \text{entero}, \text{num} \rangle$
 else
 Error(21)

G18: $\text{num} = \text{num} \times 10^{-\text{cont}}$
 if $\text{num} < \text{NUM_MAX}$ **then**
 Generar Token $\langle \text{real}, \text{num} \rangle$
 else
 Error(24)

G19: Generar Token $\langle \text{puntoComa}, - \rangle$

L: Leer el siguiente caracter

C: $\text{lexema} = \text{caracter}$ (Inicializa el buffer del lexema con el carácter actual)

C': $\text{lexema} = \text{lexema} + \text{caracter}$ (Concatena el carácter actual al final del lexema)

C'': $\text{lexema} = ''$; $\text{cont} = 0$ (Limpia el lexema acumulado y reinicia el contador de longitud)

N: $\text{lexema} = \text{lexema} + '\n'$

1.5. Errores

El analizador léxico detecta y gestiona las siguientes anomalías:

- Aparición de un carácter desconocido que no pertenece al alfabeto del lenguaje.
- Uso incorrecto o inacabado de comentarios (del tipo `//`).
- Uso incompleto del operador lógico **and** (un solo `&`).
- Falta de terminación lógica de las cadenas de texto antes de un salto de línea.
- Uso de secuencias de escape no válidas en literales de cadena.
- Números reales mal formados (ej. falta de parte decimal tras el punto).
- Exceso de tamaño para los literales numéricos o exceso de longitud en cadenas.

2. Diseño del analizador sintáctico

2.1. Gramática

Para el desarrollo del analizador sintáctico se ha diseñado una gramática en contexto formalizada para un análisis descendente tabular ($LL(1)$), definida mediante la siguiente cuádrupla:

El conjunto de símbolos no terminales (V_N) es:

$$V_N = \{A, B, C, D, E, E', F, G, H, I, K, L, O, P, Q, R, R', S, T, U, U', V, W, X, Z\}$$

El conjunto de símbolos terminales (V_T) corresponde a los tokens identificados por el analizador léxico:

$V_T = \{\text{id, entero, real, cadena, true, false, let, int, float, boolean, string, void, if, else, function, read, write, return, \&\&, >, >=, +, -, !, \dots}\}$

El axioma inicial es:

$$\text{Axioma} = P$$

El conjunto completo de producciones (P) se numera secuencialmente a continuación:

- | | |
|--|--|
| (1) $E \rightarrow RE'$ | (32) $L \rightarrow EQ$ |
| (2) $E' \rightarrow \&\&RE'$ | (33) $L \rightarrow \lambda$ |
| (3) $E' \rightarrow \lambda$ | (34) $Q \rightarrow, EQ$ |
| (4) $R \rightarrow UR'$ | (35) $Q \rightarrow \lambda$ |
| (5) $R' \rightarrow > UR'$ | (36) $X \rightarrow E$ |
| (6) $R' \rightarrow >= UR'$ | (37) $X \rightarrow \lambda$ |
| (7) $R' \rightarrow \lambda$ | (38) $B \rightarrow \text{if } (E)Z$ |
| (8) $U \rightarrow VU'$ | (39) $Z \rightarrow S$ |
| (9) $U' \rightarrow +VU'$ | (40) $Z \rightarrow \{C\}I$ |
| (10) $U' \rightarrow -VU'$ | (41) $I \rightarrow \text{else } D$ |
| (11) $U' \rightarrow \lambda$ | (42) $I \rightarrow \lambda$ |
| (12) $V \rightarrow +W$ | (43) $D \rightarrow \text{if } (E)\{C\}I$ |
| (13) $V \rightarrow -W$ | (44) $D \rightarrow \{C\}$ |
| (14) $V \rightarrow !W$ | (45) $B \rightarrow \text{let } T \text{ id};$ |
| (15) $V \rightarrow - - W$ | (46) $B \rightarrow S$ |
| (16) $V \rightarrow W$ | (47) $T \rightarrow \text{int}$ |
| (17) $W \rightarrow \text{id } O$ | (48) $T \rightarrow \text{float}$ |
| (18) $W \rightarrow (E)$ | (49) $T \rightarrow \text{boolean}$ |
| (19) $W \rightarrow \text{entero}$ | (50) $T \rightarrow \text{string}$ |
| (20) $W \rightarrow \text{real}$ | (51) $F \rightarrow \text{function } H \text{ id } (A)\{C\}$ |
| (21) $W \rightarrow \text{cadena}$ | (52) $H \rightarrow T$ |
| (22) $W \rightarrow \text{true}$ | (53) $H \rightarrow \text{void}$ |
| (23) $W \rightarrow \text{false}$ | (54) $A \rightarrow T \text{ id } K$ |
| (24) $O \rightarrow (L)$ | (55) $A \rightarrow \text{void}$ |
| (25) $O \rightarrow \lambda$ | (56) $K \rightarrow, T \text{ id } K$ |
| (26) $S \rightarrow \text{id } G;$ | (57) $K \rightarrow \lambda$ |
| (27) $S \rightarrow \text{write } E;$ | (58) $C \rightarrow BC$ |
| (28) $S \rightarrow \text{read id};$ | (59) $C \rightarrow \lambda$ |
| (29) $S \rightarrow \text{return } X;$ | (60) $P \rightarrow BP$ |
| (30) $G \rightarrow = E$ | (61) $P \rightarrow FP$ |
| (31) $G \rightarrow (L)$ | (62) $P \rightarrow \lambda$ |

2.2. Tabla Predictiva y Demostración $LL(1)$

Para la validación formal del analizador sintáctico, se calcularon los conjuntos de *First* (Primero) y *Follow* (Siguiente) de cada regla de producción. La tabla resultante demuestra empíricamente que la gramática planteada es estrictamente $LL(1)$, debido a la total ausencia de conflictos de tipo desplazamiento-reducción o reducción-reducción; es decir, no coincide más de una regla sintáctica en ninguna de las casillas predictivas.

Se adjunta la tabla al final de esta sección.

2.3. Gestión de Errores Sintácticos

El analizador sintáctico realiza un control de flujo estructurado para interceptar y registrar anomalías gramaticales sin abortar el proceso global. Conceptualmente, las inconsistencias se clasifican en dos naturalezas:

1. **Discordancia de Terminales:** Se produce cuando el símbolo en la cima de la pila de derivación es un terminal concreto, pero no se corresponde lógicamente con el token proporcionado en ese instante por el analizador léxico.
2. **Celdas Vacías en la Tabla Predictiva:** Ocurre cuando la cima de la pila contiene un símbolo no terminal, pero al cruzarlo con el token actual del flujo de entrada, la matriz predictiva devuelve una transición indeterminada o nula (*Nothing*), denotando una estructura de sentencia inválida.

Cuadro 1: Tabla de Análisis Sintáctico

[illegible]

3. Diseño del analizador semántico

Para el diseño del analizador semántico se ha definido el siguiente Esquema de Traducción (EdT), detallando las acciones semánticas asociadas a cada regla de producción para la comprobación de tipos y generación de la tabla de símbolos.

#	Producción	Acciones Semánticas
1	$E \rightarrow R E_1$	$E.tipo = \text{if}(E_1.tipo == \text{void} \vee R.tipo == E_1.tipo)$ then $R.tipo$ else tipo_error {AUX -= 2}
2	$E_1 \rightarrow \&\& R E_{1_1}$	$E_1.tipo = \text{if}(R.tipo == \text{logico} \wedge E_{1_1}.tipo \in \{\text{logico}, \text{void}\})$ then $R.tipo$ else tipo_error {AUX -= 3}
3	$E_1 \rightarrow \lambda$	$E_1.tipo = \text{void}$ {AUX -= 0}
4	$R \rightarrow U R_1$	$R.tipo = \text{if}(R_1.tipo == \text{void})$ then $U.tipo$ else if($U.tipo == R_1.tipo$) then logico else tipo_error {AUX -= 2}
5	$R_{1_1} \rightarrow > U R_{1_2}$	$R_{1_1}.tipo = \text{if}(U.tipo \in \{\text{real}, \text{entero}\} \wedge R_{1_2} == \text{void})$ then $U.tipo$ else tipo_error {AUX -= 3}
6	$R_{1_1} \rightarrow \geq U R_{1_2}$	$R_{1_1}.tipo = \text{if}(U.tipo \in \{\text{real}, \text{entero}\} \wedge R_{1_2} == \text{void})$ then $U.tipo$ else tipo_error {AUX -= 3}
7	$R_1 \rightarrow \lambda$	$R_1.tipo = \text{void}$ {AUX -= 0}
8	$U \rightarrow V U_1$	$U.tipo = \text{if}(U_1.tipo == \text{void} \vee V.tipo == U_1.tipo)$ then $V.tipo$ else tipo_error {AUX -= 2}
9	$U_1 \rightarrow + V U_{1_1}$	$U_1.tipo = \text{if}((V.tipo \in \{\text{entero}, \text{real}\} \wedge U_{1_1}.tipo == \text{void}) \vee V.tipo == U_{1_1}.tipo)$ then $V.tipo$ else tipo_error {AUX -= 3}
10	$U_1 \rightarrow - V U_{1_1}$	$U_1.tipo = \text{if}((V.tipo \in \{\text{entero}, \text{real}\} \wedge U_{1_1}.tipo == \text{void}) \vee V.tipo == U_{1_1}.tipo)$ then $V.tipo$ else tipo_error {AUX -= 3}
11	$U_1 \rightarrow \lambda$	$U_1.tipo = \text{void}$ {AUX -= 0}
12	$V \rightarrow + W$	$V.tipo = \text{if}(W.tipo \in \{\text{entero}, \text{real}\})$ then $W.tipo$ else tipo_error {AUX -= 2}
13	$V \rightarrow - W$	$V.tipo = \text{if}(W.tipo \in \{\text{entero}, \text{real}\})$ then $W.tipo$ else tipo_error {AUX -= 2}

Continúa en la siguiente página...

#	Producción	Acciones Semánticas
14	$V \rightarrow ! W$	$V.tipo = \text{if}(W.tipo == \text{lógico}) \text{ then } tipo_logico$ $\quad \text{else } tipo_error$ $\{AUX -= 2\}$
15	$V \rightarrow - - id$	$V.tipo = \text{if}(\text{buscarTipo}(id.pos) \in \{\text{real}, \text{entero}\})$ $\quad \text{then } buscarTipo(id.pos)$ $\quad \text{else } tipo_error$ $\{AUX -= 2\}$
16	$V \rightarrow W$	$V.tipo = W.tipo$ $\{AUX -= 1\}$
17	$W \rightarrow id O$	$W.tipo = \text{if}(O.tipo == \text{void}) \text{ then } buscarTipo(id.pos)$ $\quad \text{else if}(\text{buscarParam}(id.pos) == O.parametros)$ $\quad \quad \text{then } buscarTipoRet(id.pos)$ $\quad \quad \text{else } tipo_error$ $\{AUX -= 2\}$
18	$W \rightarrow (E)$	$W.tipo = E.tipo$ $\{AUX -= 3\}$
19	$W \rightarrow \text{entero}$	$W.tipo = \text{entero} \quad \{AUX -= 1\}$
20	$W \rightarrow \text{real}$	$W.tipo = \text{real} \quad \{AUX -= 1\}$
21	$W \rightarrow \text{cadena}$	$W.tipo = \text{cadena} \quad \{AUX -= 1\}$
22	$W \rightarrow \text{true}$	$W.tipo = \text{lógico} \quad \{AUX -= 1\}$
23	$W \rightarrow \text{false}$	$W.tipo = \text{lógico} \quad \{AUX -= 1\}$
24	$O \rightarrow (L)$	$O.tipo = tipo_funcion$ $O.parametros = L.parametros$ $O.numParam = L.numeroParametros$ $\{AUX -= 3\}$
25	$O \rightarrow \lambda$	$O.tipo = \text{void}$ $\{AUX -= 0\}$
26	$S \rightarrow id G ;$	$S.tipo = \text{if}(\text{buscarTipo}(id.pos) == G.tipo == tipo_funcion)$ $\quad \text{then if}(\text{buscarParam}(id.pos) \neq$ $G.parametros) \text{ tipo_error}$ $\quad \text{else if}(\text{buscarTipo}(id.pos) \neq G.tipo) \text{ then } tipo_error$ $\{AUX -= 3\}$
27	$S \rightarrow \text{write } E ;$	$S.tipo = \text{if}(E.tipo \notin$ $\{\text{entero}, \text{real}, \text{cadena}\}) \text{ then } tipo_error$ $S.tipoRet = \text{void}$ $\{AUX -= 3\}$
28	$S \rightarrow \text{read } id ;$	$S.tipo = \text{if}(\text{buscarTipoTS}(id.pos) \notin \{\text{entero}, \text{real}, \text{cadena}\})$ $\quad \text{then } tipo_error$ $S.tipoRet = \text{void}$ $\{AUX -= 3\}$
29	$S \rightarrow \text{return } X ;$	$S.tipo = \text{if}(X.tipo \neq S.tipoRet) \text{ then } tipo_error$ $\{AUX -= 3\}$
30	$G \rightarrow = E$	$G.tipo = E.tipo$ $\{AUX -= 2\}$
31	$G \rightarrow (L)$	$G.tipo = tipo_funcion$ $G.parametros = L.parametros, G.numParam =$ $L.numParametros$ $\{AUX -= 3\}$
Continúa en la siguiente página...		

#	Producción	Acciones Semánticas
32	$L \rightarrow E Q$	$L.param = E.tipo \times Q.param, L.numP = 1 + Q.numP$ {AUX -= 2}
33	$L \rightarrow \lambda$	$L.param = \text{void}, L.numP = 0$ {AUX -= 0}
34	$Q_1 \rightarrow, E Q_2$	$Q_1.param = E.tipo \times Q_2.param, Q_1.numP = 1 + Q_2.numP$ {AUX -= 3}
35	$Q \rightarrow \lambda$	$Q.param = \text{void}, Q.numP = 0$ {AUX -= 0}
38, 39	$B \rightarrow \text{if} (E) Z$	$B.tipo = \text{if}(E.tipo == \text{logico}) \text{ then } tipo_error$ $B.tipoRet = Z.tipoRet$ {AUX -= 5}
51, 52	$B \rightarrow \text{let } T \text{ id} ;$	$zonaDecl = \text{true}$ añadirTipo(id.pos , $T.tipo$) añadirDespl(id.pos , $despl$) $desplazamiento+ = T.tamaño$ $zonaDecl = \text{false}$ {AUX -= 4}
58-60	$F \rightarrow \text{function } H \text{ id} (A) \{ C \}$	$zonaDecl = \text{true}$, añadeTipo(id.pos , $tipo_func$) añadeTipoRet(id.pos , $H.tipo$), $C.tipoRet = H.tipo$ añadeEtiq(id.pos , $nuevaEtiq()$) $TSF = \text{crearTabla}()$, $despl = 0$, $zonaDecl = \text{false}$ añadeParam(id.pos , $A.param$), $liberarTabla(TSF)$ {AUX -= 9}
68, 69	$PP \rightarrow P$	$TS = \text{crearTabla}()$, $desplazamiento = 0$ $liberarTabla(TS)$ {AUX -= 1}

3.1. Errores

El analizador semántico detecta y gestiona los siguientes errores de tipado e incoherencias de contexto:

- Uso de tipos incompatibles en expresiones lógicas y relacionales (ej. operadores **and**, **>**, **>=**).
- Operaciones aritméticas sin conversión implícita de tipos (mezcla de enteros y reales sin coerción explícita).
- Invocación de funciones con número o tipo incorrecto de parámetros en comparación con su declaración.
- Asignación de valores a variables de distinto tipo.
- Utilización de las sentencias **write** y **read** con variables de tipos no permitidos (solo se admite entero, real o cadena).
- Discordancia entre el tipo devuelto por la sentencia **return** y el tipo de retorno declarado en la firma de la función.
- Evaluación de sentencias de control (como el condicional **if**) donde la expresión interna no se resuelve a un tipo lógico.

4. Diseño de la Tabla de Símbolos

Para la gestión de los identificadores del programa se ha diseñado una Tabla de Símbolos estructurada en dos niveles de ámbito: global y local. Esta decisión permite dar soporte a la declaración de variables dentro y fuera de las funciones, así como al sombreado de variables.

4.1. Organización y Ámbitos

- **Tabla Global:** Almacena las variables declaradas en el flujo principal del programa, así como las firmas de las funciones. Sus elementos se indexan lógicamente con valores enteros positivos (0, 1, 2, ...).
- **Tabla Local:** Se crea de forma dinámica al entrar en el contexto de una función (tras leer la palabra reservada `function`) y se destruye al salir de la misma. Para diferenciar sus entradas de las globales de forma eficiente, los identificadores locales se indexan mediante valores enteros negativos (-1, -2, -3, ...).

4.2. Estructura de la Información

La política de acceso y resolución de nombres se basa en una búsqueda lineal, consultando siempre en primer lugar el ámbito local (si existe) y, de no encontrar coincidencia, procediendo al ámbito global.

Cada registro en la tabla almacena los siguientes atributos actualizados por el analizador semántico:

- **Lexema:** La cadena de texto que da nombre al identificador.
- **Tipo de dato:** Entero, real, lógico, cadena, void o función.
- **Desplazamiento (*Offset*):** Dirección relativa de memoria calculada a partir del tamaño en bytes del tipo de dato asociado (ej. entero = 2 bytes, real = 4 bytes, cadena = 64 bytes).
- **Atributos de función:** Si el identificador corresponde a una función, se almacena adicionalmente su tipo de retorno, el número total de parámetros esperados y una lista ordenada con los tipos exactos de dichos parámetros.

5. Diseño del Gestor de Errores

El gestor de errores es un módulo transversal invocado por los analizadores léxico, sintáctico y semántico ante cualquier anomalía en el código fuente.

5.1. Seguimiento de Líneas

Para proporcionar diagnósticos precisos que permitan al programador localizar los fallos, el procesador mantiene un contador de línea actual. Este contador es incrementado exclusivamente por el analizador léxico cada vez que detecta el delimitador de salto de línea. Cuando el gestor registra un error, este se asocia de manera automática a la línea en curso.

5.2. Estrategia de Recuperación de Errores

Se ha tomado la decisión de diseño de implementar un mecanismo de recuperación de errores en lugar de abortar la compilación o detener el programa ante el primer fallo detectado. El objetivo es analizar la totalidad del fichero fuente y emitir un informe completo en una sola pasada.

El funcionamiento lógico se estructura de la siguiente manera:

- **Detección y Registro:** Cuando un analizador detecta una estructura inválida (ej. un carácter extraño, un token no esperado o un tipo semántico incompatible), se genera una alerta con su código de error, descripción y línea. Esta alerta se concatena en el registro interno del gestor.
- **Sincronización:** En lugar de colapsar, el sistema descarta la cadena conflictiva o reinicia las estructuras de análisis temporales (como la pila sintáctica o el lexema acumulado) hasta encontrar el siguiente punto de sincronización lógico y seguro (por ejemplo, un delimitador de final de sentencia ; o el cierre de un bloque }). Esto permite retomar la evaluación del resto del código.
- **Inconvenientes (Errores en Cascada):** Como contrapartida a la recuperación sistemática, esta técnica genera en ocasiones errores derivados o falsos positivos. Por ejemplo, si una cadena de texto no se cierra correctamente con comillas dobles, el analizador léxico absorberá el resto de caracteres de su misma línea, lo que provocará casi con seguridad que el analizador sintáctico emita errores por falta de terminadores lógicos (como el punto y coma) que quedaron engullidos. Esta es una decisión consciente y un estándar adoptado por los compiladores modernos, asumiendo que informar de múltiples fallos útiles compensa la aparición ocasional de falsos positivos en cascada.